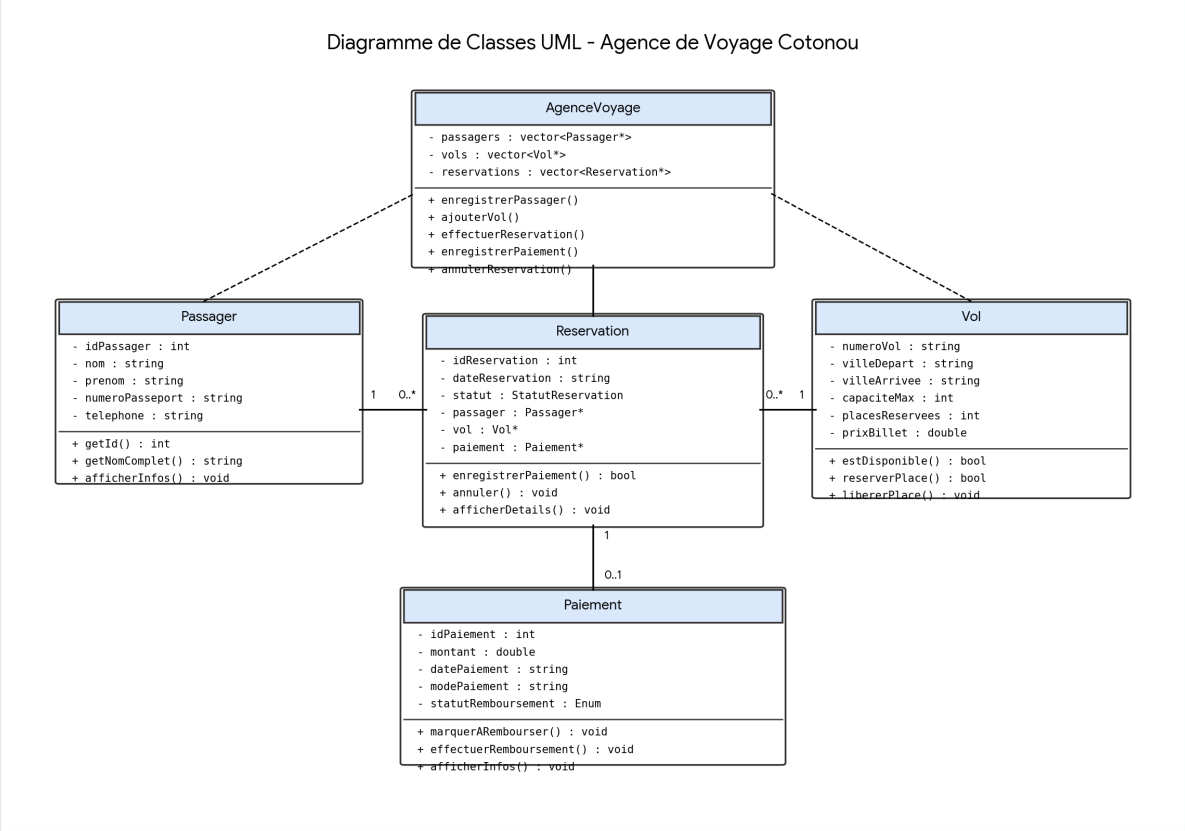
Dossier UML Complet - Agence de Voyage (Cotonou)

Ce document regroupe en un seul fichier les **4 éléments demandés** : les deux codes sources PlantUML (`.puml`) et les deux rendus visuels des diagrammes (`.png`).

1. Diagramme de Classes — Rendu Visuel `diagramme_classes.png`



2. Diagramme de Classes — Code Source PlantUML

`diagramme_classes.puml`

Copiez le code ci-dessous dans **PlantUML** ou **Lucidchart** (*Insérer* > *Diagramme UML* > *PlantUML*) pour éditer le schéma :

```
@startuml
skinparam classAttributeIconSize 0
skinparam linetype ortho

enum StatutReservation {
  EN_ATTENTE
  CONFIRMEE
  ANNULEE
}

enum StatutRemboursement {
  NON_APPLICABLE
  A_REMBOURSER
  REMBOURSE
}

class Passager {
  - idPassager : int
  - nom : string
  - prenom : string
  - numeroPasseport : string
  - telephone : string
  + Passager(id, nom, prenom, passeport, tel)
  + getId() : int
  + getNomComplet() : string
  + afficherInfos() : void
}

class Vol {
  - numeroVol : string
  - villeDepart : string
  - villeArrivee : string
  - dateHeure : string
  - capaciteMax : int
  - placesReservees : int
  - prixBillet : double
  + Vol(num, dep, arr, date, cap, prix)
  + getNumeroVol() : string
  + getPlacesDisponibles() : int
  + estDisponible() : bool
  + reserverPlace() : bool
  + libererPlace() : void
  + afficherInfos() : void
}

class Paiement {
  - idPaiement : int
  - montant : double
  - datePaiement : string
  - modePaiement : string
  - statutRemboursement : StatutRemboursement
  + Paiement(id, montant, date, mode)
  + marquerARembourser() : void
  + effectuerRemboursement() : void
  + afficherInfos() : void
}

class Reservation {
```

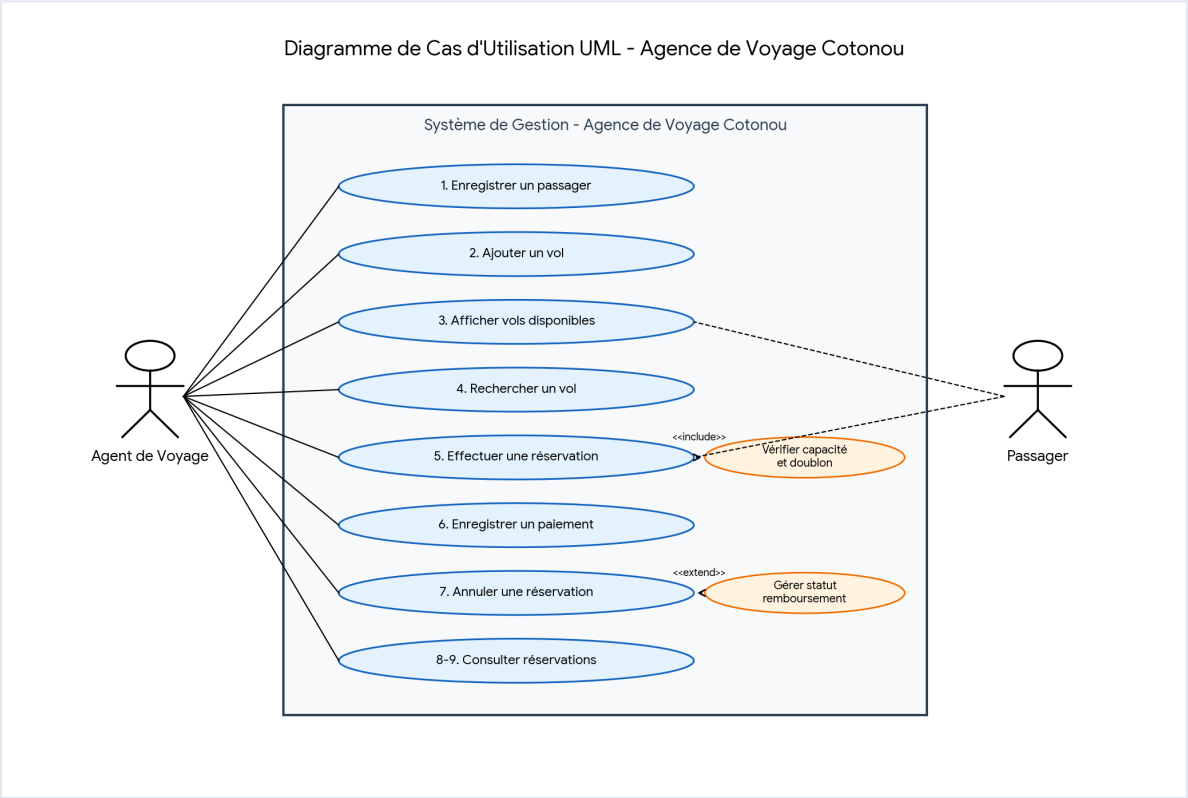
```
  - idReservation : int
  - dateReservation : string
  - statut : StatutReservation
  - passager : Passager*
  - vol : Vol*
  - paiement : Paiement*
  + Reservation(id, date, p, v)
  + getId() : int
  + getStatut() : StatutReservation
  + enregistrerPaiement(id, date, mode) : bool
  + annuler() : void
  + afficherDetails() : void
}

class AgenceVoyage {
  - passagers : vector
  - vols : vector
  - reservations : vector
  + enregistrerPassager() : void
  + ajouterVol() : void
  + afficherVolsDisponibles() : void
  + rechercherVol() : void
  + effectuerReservation() : void
  + enregistrerPaiement() : void
  + annulerReservation() : void
  + consulterReservations() : void
  + afficherInfoReservation() : void
}

Passager "1" -- "0..*" Reservation : effectue >
Vol "1" -- "0..*" Reservation : concerne >
Reservation "1" -- "0..1" Paiement : regle par >
AgenceVoyage "1" o-- "0..*" Passager
AgenceVoyage "1" o-- "0..*" Vol
AgenceVoyage "1" o-- "0..*" Reservation
@enduml
```

3. Diagramme de Cas d'Utilisation — Rendu Visuel

`diagramme_cas_utilisation.png`



4. Diagramme de Cas d'Utilisation — Code Source PlantUML

`diagramme_cas_utilisation.puml`

Copiez le code ci-dessous dans **PlantUML** ou **Lucidchart** pour éditer le diagramme de cas d'utilisation :

```
@startuml
left to right direction
skinparam packageStyle rectangle

actor "Agent de Voyage" as Agent
actor "Passager" as Client

rectangle "Système de Gestion - Agence de Voyage Cotonou" {
  usecase "Enregistrer un passager" as UC1
  usecase "Ajouter un vol" as UC2
  usecase "Afficher les vols disponibles" as UC3
  usecase "Rechercher un vol" as UC4
  usecase "Effectuer une réservation" as UC5
  usecase "Enregistrer un paiement" as UC6
  usecase "Annuler une réservation" as UC7
  usecase "Consulter les réservations" as UC8
  usecase "Afficher les infos d'une réservation" as UC9
  usecase "Vérifier capacité et doublon" as UC10
  usecase "Gérer le remboursement" as UC11
}

Agent --> UC1
Agent --> UC2
Agent --> UC3
Agent --> UC4
Agent --> UC5
Agent --> UC6
Agent --> UC7
Agent --> UC8
Agent --> UC9

Client ..> UC3 : consulte
Client ..> UC5 : demande

UC5 ..> UC10 : <>
UC6 ..> UC5 : <> (valide place)
UC7 ..> UC11 : <> (si déjà payée)
@enduml
```